

Mount Kenya University

Unlocking Infinite Possibilities

- Name: **BRIAN KIPKORIR KIPTUI**
- Admission Number: **BSCCS/2024/53968**
- Course: **COMPUTER SCIENCE**
- Unit Code: **BIT3208**
- Unit Name: **Advanced Web Design and Development**
- Lecturer Name: **MICHAEL NYORO**
- Academic Year: **YEAR III SEM II**

Project Details

Project Title

Web-Based Academic Resource Sharing System

A Web-Based Academic Resource Sharing System is an online platform designed to help students and lecturers upload, share, manage, and access academic materials through a centralized web application.

The system allows users to:

- upload notes
- share assignments
- access revision materials
- download learning resources
- manage academic documents online

Selected Technologies

.Js + MySQL+html

I pick these because they are simple to use.

GitHub Link

<https://github.com/Brian900bkk/AcademicResourceSystem.git>

Table of Content

WEEK 1: LOCAL ENVIRONMENT SETUP.....	1
Figure 1. Apache and MySQL Running.....	1
Figure 2. Localhost Test Page.....	1
Fig 3: Hello World Test.....	1
Fig 4: Database Connection Test.....	1
WEEK 2: WIREFRAMES AND GUI DESIGN.....	2
Fig 1: Hand-drawn Wireframe.....	2
Fig 2: Dashboard Layout Design.....	2
Fig 3: Mobile View Mockup.....	2
Fig 4: Color Theme and Navigation Structure.....	2
FIG 5: GUI PROTOTYPE IN FIGMA.....	2
WEEK 3.....	3
Fig 1: JavaScript Form Validation.....	3
Fig 2: Password Strength Checker.....	3
Fig 3: PHP Syntax Practice.....	3
Fig 4: Database Connection Script.....	3
Fig 5: Dynamic User Input Handling.....	3
WEEK 4.....	4
Fig1: Login form.....	4
Fig 2: JavaScript Form	4
Fig 3: Folder structure.....	4
FIG 4: Database connection.....	4
Fig5 : GitHub weekly Update.....	4
WEEK 5	5
FIG1: USERS TABLE.....	5
FIG2:USER EMAIL AND DECRYPTED PASSWORD.....	5
FIG3: DISPLAY ALL USER.....	5
FIG4: CREATION OF USER TABLE.....	5
FIG5: INSERTION IN TABLE.....	5

Week 1: Local Environment Setup

Fig 1: Installation of XAMPP



Fig 2: Apache and MySQL Running

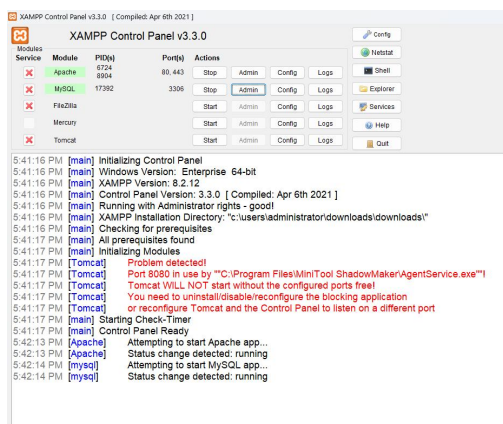


Fig 3: Localhost Test Page

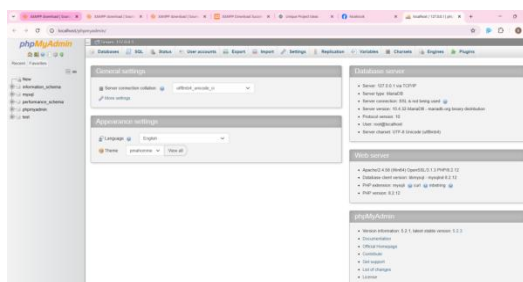
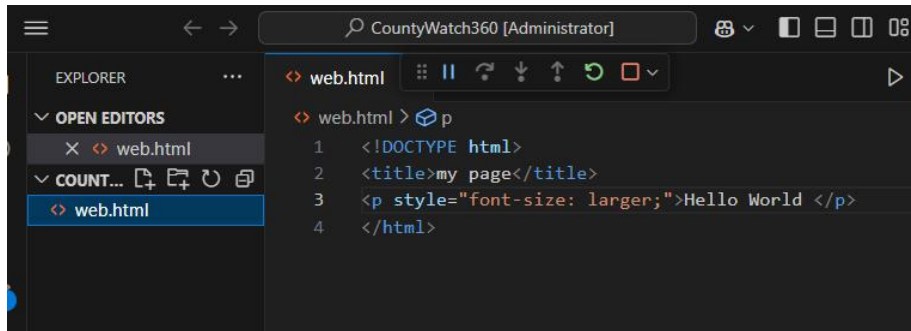


Fig 4: code testing

Simple code



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a file named 'web.html' under the 'COUNT...' folder. The main editor area displays the content of 'web.html', which is a simple HTML document with a title 'my page' and a paragraph 'Hello World' in a larger font size.

```
web.html > p
1 <!DOCTYPE html>
2 <title>my page</title>
3 <p style="font-size: larger;">Hello World </p>
4 </html>
```

Results

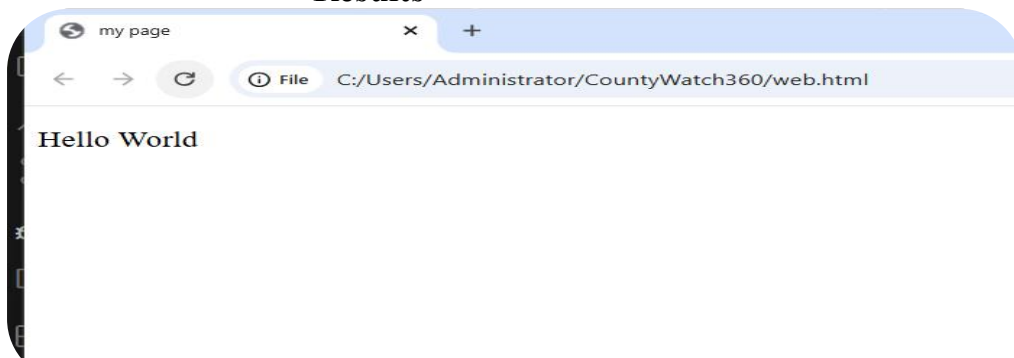
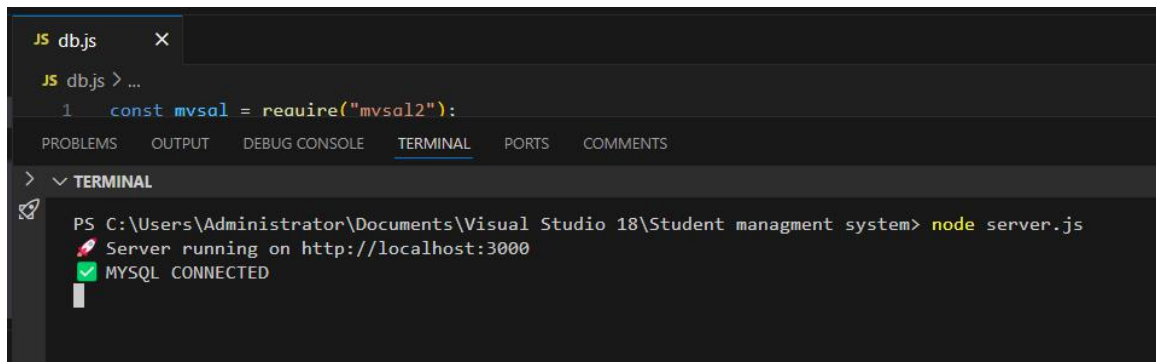


Fig 5: Database Connection Test



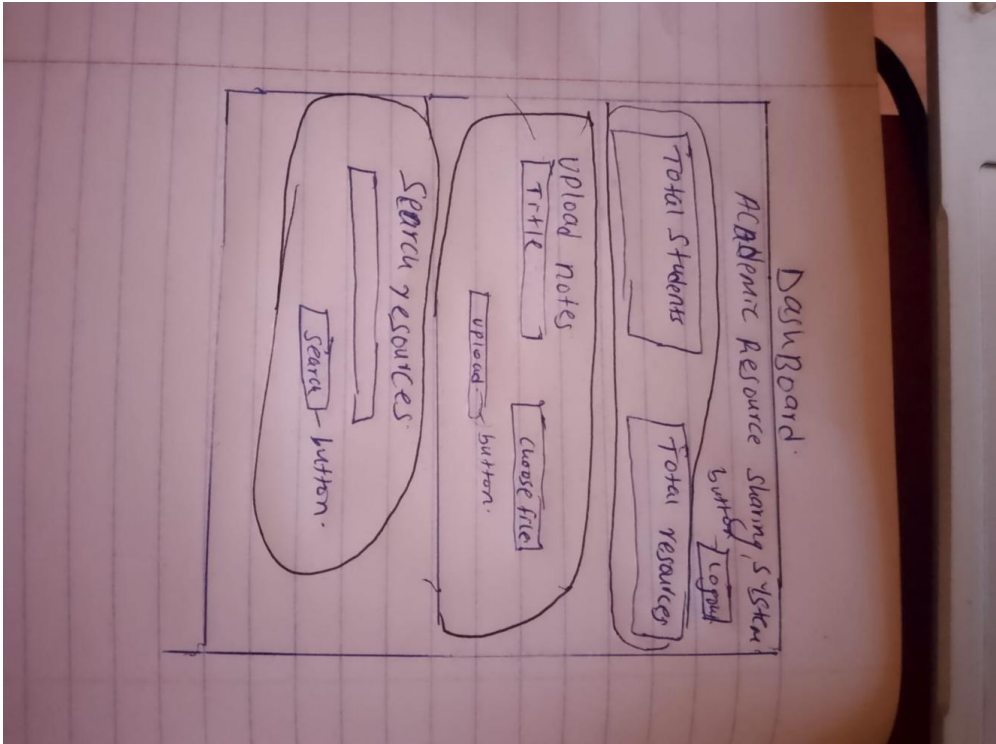
The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the command 'node server.js' and the output 'Server running on http://localhost:3000' and 'MYSQL CONNECTED'.

```
PS C:\Users\Administrator\Documents\Visual Studio 18\Student managment system> node server.js
Server running on http://localhost:3000
MYSQL CONNECTED
```

WEEK 2: Wireframes and GUI Design

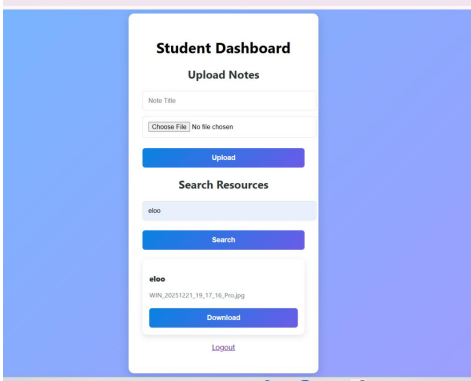
User Interface Planning and System Design

Fig 1: Hand-drawn Wireframe



It outlines the arrangement of key pages including the login page, dashboard, notes management section, announcements area, and administration panel. The wireframe provides a clear overview of the system's intended structure and navigation flow before development begins.

Fig 2: Dashboard Layout Design



this the working dashboard of the system

Fig 3: Mobile View Mockup

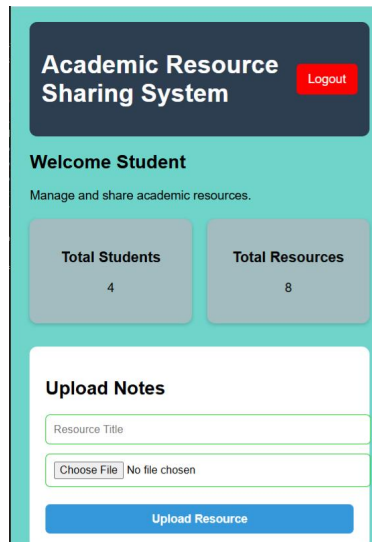
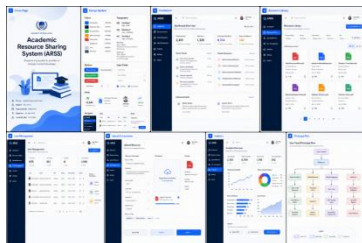


Fig 4: Color Theme and Navigation Structure

This illustrates the established visual aesthetics, contrasting colors, and navigation flow designed to ensure a clean and consistent user experience

Fig 5: GUI Prototype in Figma/Canva/Draw.io



The prototype provides a visual representation of how users will interact with the system before actual development begins. It illustrates the layout of key components such as navigation menus, login forms, dashboards, buttons, and content sections.

Week 3: JavaScript and PHP Basics

Fig 1: JavaScript Form Validation

```

document.getElementById("registerForm").addEventListener("submit", async (e) => {
    e.preventDefault();
    const name = document.getElementById("name").value;
    const email = document.getElementById("registerEmail").value;
    const password = document.getElementById("registerPassword").value;

    const response = await fetch("/register", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, email, password })
    });

    const result = await response.json();
    alert(result.message);
    if (response.ok) {
        window.location.href = "/login";
    }
});

```

Use JavaScript to check:

Required fields are filled.

Email format is valid.

Passwords match

Fig 2: Password Strength Checker

```

<h1>Login</h1>
<form id="loginForm">
    <input type="email" id="loginEmail" placeholder="Email" required>
    <input type="password" id="loginPassword" placeholder="Password" required onkeyup="checkStrength(this.value)">
    <div id="strengthMessage"></div>
    <button type="submit">Login</button>
</form>

```

- Use JavaScript to check password strength (length, uppercase, numbers, symbols).

Fig 3: PHP Syntax Practice

```

<?php
$name = "Brian";
$numbers = [1, 2, 3, 4];

foreach ($numbers as $num) {
    echo "Number: $num <br>";
}

function greet($name) {
    return "Hello, $name!";
}

echo greet($name);
?>

```

It shows:

- Variables
- Arrays
- Loops
- Functions

Fig 4: Database Connection Script

```
25 const db = mysql.createPool({
26   host: "localhost",
27   user: "root",
28   password: "",
29   database: "academic_resources_system",
30   port: 3306,
31   waitForConnections: true,
32   connectionLimit: 10,
33   queueLimit: 0
34 });
35 db.getConnection((err, connection) => {
36   if (err) {
37     console.error("Database connection failed:", err.message);
38     process.exit(1);
39   }
40   console.log("Connected to MySQL database.");
41   connection.release();
42 });
43
```

Show a PHP script connecting to MySQL.

Fig 5: Dynamic User Input Handling

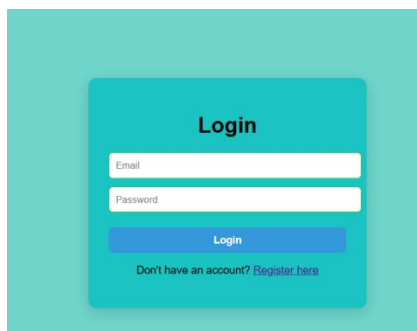
```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];

    echo "Welcome, $name! Your email is $email.";
}
?>
```

Show PHP handling form input (e.g., saving registration data).

Week 4:

Fig 1:login form



This shows the user-facing graphical interface built for authentication, providing clean input fields for standard user identification.

Fig 2:form validation

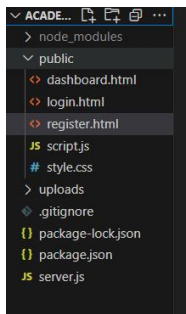
```
document.getElementById("registerForm").addEventListener("submit", async (e) => {
    e.preventDefault();
    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("registerEmail").value.trim();
    const password = document.getElementById("registerPassword").value;
    const confirmPassword = document.getElementById("confirmPassword").value;

    const messageDiv = document.getElementById("registerMessage");

    // Custom validation
    if (name.length < 3) {
        messageDiv.textContent = "Name must be at least 3 characters.";
        messageDiv.style.color = "red";
        return;
    }
    if (!email.includes("@") || !email.includes(".")) {
        messageDiv.textContent = "Please enter a valid email address.";
        messageDiv.style.color = "red";
        return;
    }
    if (password.length < 6) {
        messageDiv.textContent = "Password must be at least 6 characters.";
        messageDiv.style.color = "red";
        return;
    }
    if (confirmPassword !== password) {
        messageDiv.textContent = "Passwords do not match.";
        messageDiv.style.color = "red";
        return;
    }
    // If all validations pass, proceed with registration logic
    // ...
});
```

The figure above demonstrates the form validation mechanism, where the system checks user input and displays appropriate error messages when invalid or incomplete data is entered. This ensures that only valid data is submitted and stored in the system.

Fig 3:Folder structure



The structure separates frontend resources, backend logic, database files, and uploaded content to improve maintainability, scalability, and ease of development.

WEEK 5:

Fig1:Database screenshot

Table	Action	Rows	Type	Collation	Size	Overhead
☐ notes	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ resources	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ students	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
3 tables	Sum	2	InnoDB	utf8mb4_general_ci	80.0 KiB	0 B

☐ Check all With selected:

It contains the students email,password and name while the resources contain notes or learning materials

Fig 2: Table creation

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	id			No	None		AUTO_INCREMENT	Change Drop More
☐	2	name			No	None			Change Drop More
☐	3	email			No	None			Change Drop More
☐	4	password			No	None			Change Drop More

shows how the registration table looks like

Fig3: Login database integration

name	varchar(255)	utf8mb4_general_ci	No	None	Change Drop More
email	varchar(255)	utf8mb4_general_ci	No	None	Change Drop More
password	varchar(255)	utf8mb4_general_ci	No	None	Change Drop More

The figure shows the login authentication process where user credentials are validated against database records before granting access to the system. This integration ensures secure user authentication and access control.

Fig 4: SQL query

Student table

```
CREATE TABLE students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL  
);
```

The figure shows the SQL query used to retrieve student information from the Students table. The query displays details such as student ID, name, email address, course, and other academic information stored in the database.

Resource table

```
CREATE TABLE resources (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  title VARCHAR(255),
  filename VARCHAR(255) NOT NULL,
  original_name VARCHAR(255) NOT NULL,
  FOREIGN KEY (student_id) REFERENCES students(id)
);
```

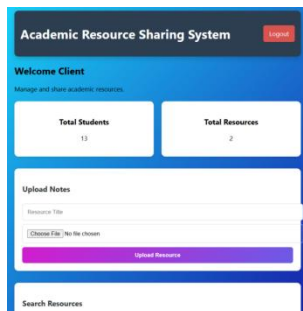
The figure shows the

SQL query used to retrieve learning resources stored in the database. The table contains information such as resource title, file name, upload date, category, and the user who uploaded the resource.

Week 6 , 7 and 8

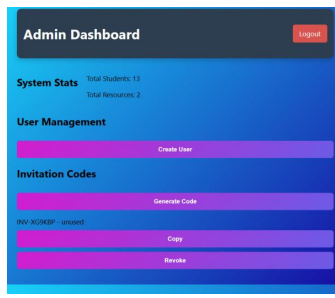
Student,manager and Admin dashboard

Student dashboard



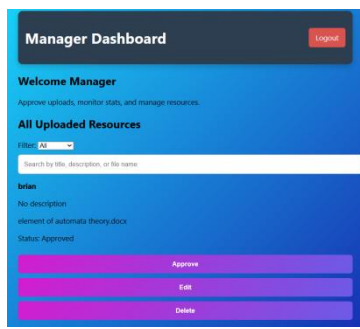
- **Upload Section** → students can add study resources or files into the system.
- **Search Bar** → allows students to quickly find learning materials by keyword.
- **Resource List** → displays all approved resources available for study.
- **Stats Panel** → shows simple counts like how many resources are available.
- **Profile/Logout** → gives students a way to manage their account and exit securely.

Admin dashboard



- **System Stats** → shows totals like number of students and resources in the system.
- **User List** → displays all registered users with their roles.
- **Create User Form** → lets admins add new managers or other admins.
- **Delete User Button** → removes users from the system when necessary.
- **Invitation Code Generator** → creates special codes required for manager/admin registration.
- **Code List** → shows all codes with their status (unused or used).
- **Copy Button** → allows admins to copy codes easily for sharing.
- **Revoke Button** → deletes or invalidates codes that should no longer be used.

Manager dashboard



- **Resource List** → shows all uploaded files (pending and approved).
- **Approve Button** → lets managers mark pending uploads as officially approved.
- **Edit Button** → allows managers to change resource details or replace files.
- **Delete Button** → removes inappropriate or outdated resource.

- **Search Bar** → lets managers find specific resources by title, description, or file name.